## Question 4iii

```r
# Loading the data and renaming columns
data <- read.csv("data.csv", header = FALSE)
names(data) <- c("t", "z")
```

```r
# Modelling z(t) = (y(t))^2
z_model <- function(t, C, lambda1, lambda2) {
  y <- C * (exp(lambda1 * t) - exp(lambda2 * t)) / lambda1 - lambda2
  y^2
}
```

```r
# Finding appropriate start values
data_pos_vals <- subset(data, z > 0)

lm_start <- lm(log(z) ~ t, data = data_pos_vals)
lambda1_start <- coef(lm_start)[["t"]] / 2

start_vals <- list(C = 1, lambda1 = lambda1_start, lambda2 = -0.05)
```

```r
start_vals
```

```
## $C
## [1] 1
##
## $lambda1
## [1] 0.3441271
##
## $lambda2
## [1] -0.05
```

```r
# Nonlinear least squares procedure
# Using the Levenberg-Marquardt algorithm for robustness
# install.packages("minpack.lm")
library(minpack.lm)
```

```
## Warning: package 'minpack.lm' was built under R version 4.2.3
```

```r
nls_model <- nlsLM(z ~ z_model(t, C, lambda1, lambda2), data = data,
                   start = start_vals)
```

```r
summary(nls_model)
```

```
##
## Formula: z ~ z_model(t, C, lambda1, lambda2)
##
## Parameters:
##         Estimate Std. Error t value Pr(>|t|)
## C        0.27726    0.03669   7.557 2.86e-13 ***
## lambda1  0.21390    0.02971   7.199 3.06e-12 ***
```

```
## lambda2 -0.32796    0.16724  -1.961   0.0506 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.177 on 398 degrees of freedom
##
## Number of iterations to convergence: 9
## Achieved convergence tolerance: 1.49e-08
```

```r
# Recovering identifiable combinations
pars <- coef(nls_model)

C_hat <- pars["C"]
lambda1_hat <- pars["lambda1"]
lambda2_hat <- pars["lambda2"]

sigma_plus_gamma_hat <- -(lambda1_hat + lambda2_hat)
sigma_beta_minus_gamma_hat <- -(lambda1_hat * lambda2_hat)
```

```r
cat("Fitted identifiable parameters/combinations:\n\n")
```

```
## Fitted identifiable parameters/combinations:
```

```r
cat("lambda1 =", lambda1_hat, "\n")
```

```
## lambda1 = 0.2138953
```

```r
cat("lambda2 =", lambda2_hat, "\n")
```

```
## lambda2 = -0.3279615
```

```r
cat("sigma + gamma =", sigma_plus_gamma_hat, "\n")
```

```
## sigma + gamma = 0.1140662
```

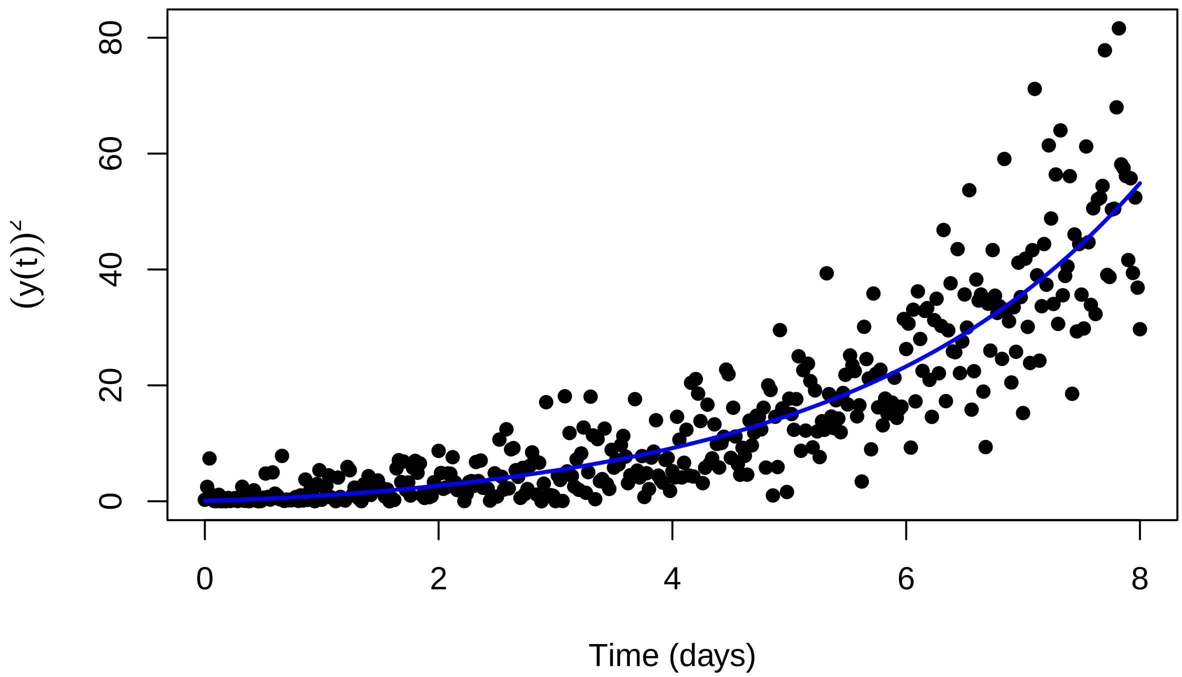
```r
cat("sigma(beta - gamma) =", sigma_beta_minus_gamma_hat, "\n")
```

```
## sigma(beta - gamma) = 0.0701494
```

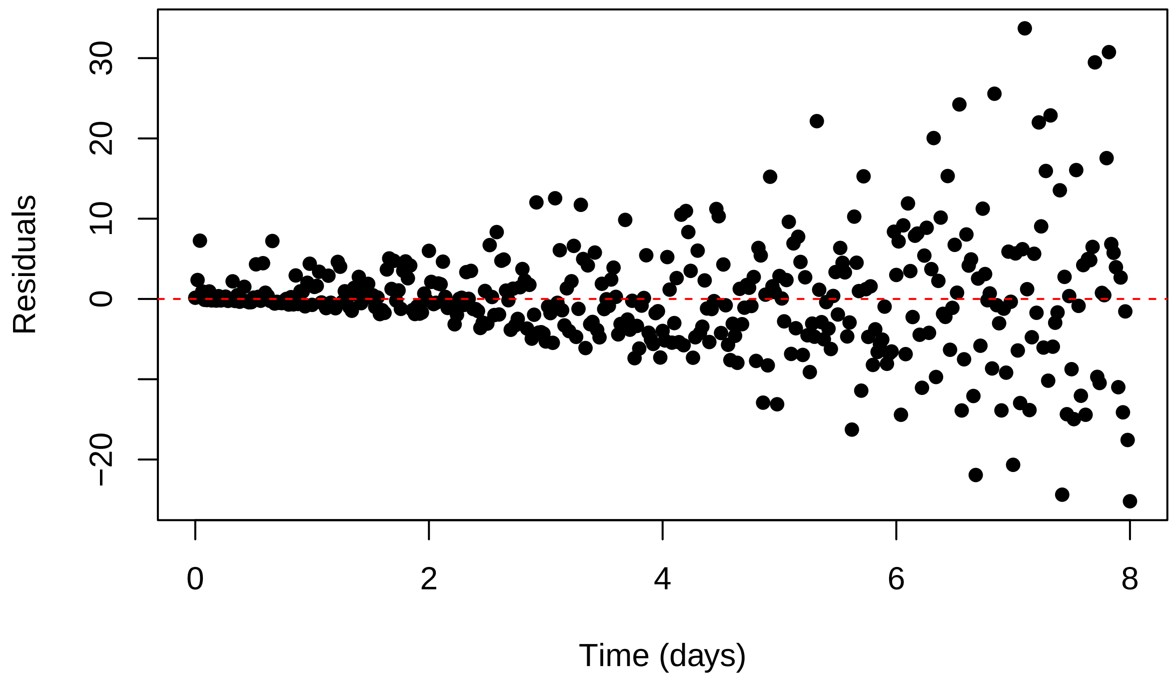
```r
# Plotting fitted values
data$z_fitted <- predict(nls_model)
data$resid <- resid(nls_model)

plot(data$t, data$z, pch = 16, xlab = "Time (days)",
     ylab = expression((y(t))^2))

lines(data$t[order(data$t)], data$z_fitted[order(data$t)], lwd = 2,
      col = "blue")
```




```r
# Plotting residuals
plot(data$t, data$resid, pch = 16, xlab = "Time (days)", ylab = "Residuals")
abline(h = 0, lty = 2, col = "red")
```




### Question 4iv

```r
# Exponential fit: g(t) = a + b * exp(c * t)
a_start <- min(data$z)

data_shift <- subset(data, z > a_start)
lm_exp_start <- lm(log(z - a_start + 1e-16) ~ t, data = data_shift)

c_start <- coef(lm_exp_start)[["t"]]
b_start <- coef(lm_exp_start)[["(Intercept)"]]
exp_start_vals <- list(a = a_start, b = b_start, c = c_start)

exp_model <- nlsLM(z ~ a + b * exp(c * t), data = data,
                   start = exp_start_vals)
```

```r
summary(exp_model)
```

```
##
## Formula: z ~ a + b * exp(c * t)
##
## Parameters:
##   Estimate Std. Error t value Pr(>|t|)
## a -1.65066    1.15334  -1.431    0.153
```

```
## b  2.06951    0.49014   4.222    3e-05 ***
## c  0.41388    0.02951  14.024   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.17 on 398 degrees of freedom
##
## Number of iterations to convergence: 8
## Achieved convergence tolerance: 1.49e-08
```

```r
# Polynomial fits
poly2_model <- lm(z ~ t + I(t^2), data = data)
poly3_model <- lm(z ~ t + I(t^2) + I(t^3), data = data)
poly4_model <- lm(z ~ t + I(t^2) + I(t^3) + I(t^4), data = data)
```

```r
# Predicted values
data$exp_fitted <- predict(exp_model)
data$poly2_fitted <- predict(poly2_model)
data$poly3_fitted <- predict(poly3_model)
data$poly4_fitted <- predict(poly4_model)
```

```r
# Comparing RSS
rss <- function(obs, fit) sum((obs - fit)^2)

rss_mech <- rss(data$z, data$z_fitted)
rss_exp <- rss(data$z, data$exp_fitted)
rss_poly2 <- rss(data$z, data$poly2_fitted)
rss_poly3 <- rss(data$z, data$poly3_fitted)
rss_poly4 <- rss(data$z, data$poly4_fitted)
```

```r
c(Mechanistic = rss_mech, Exponential = rss_exp, Quadratic = rss_poly2,
  Cubic = rss_poly3, Quartic = rss_poly4)
```

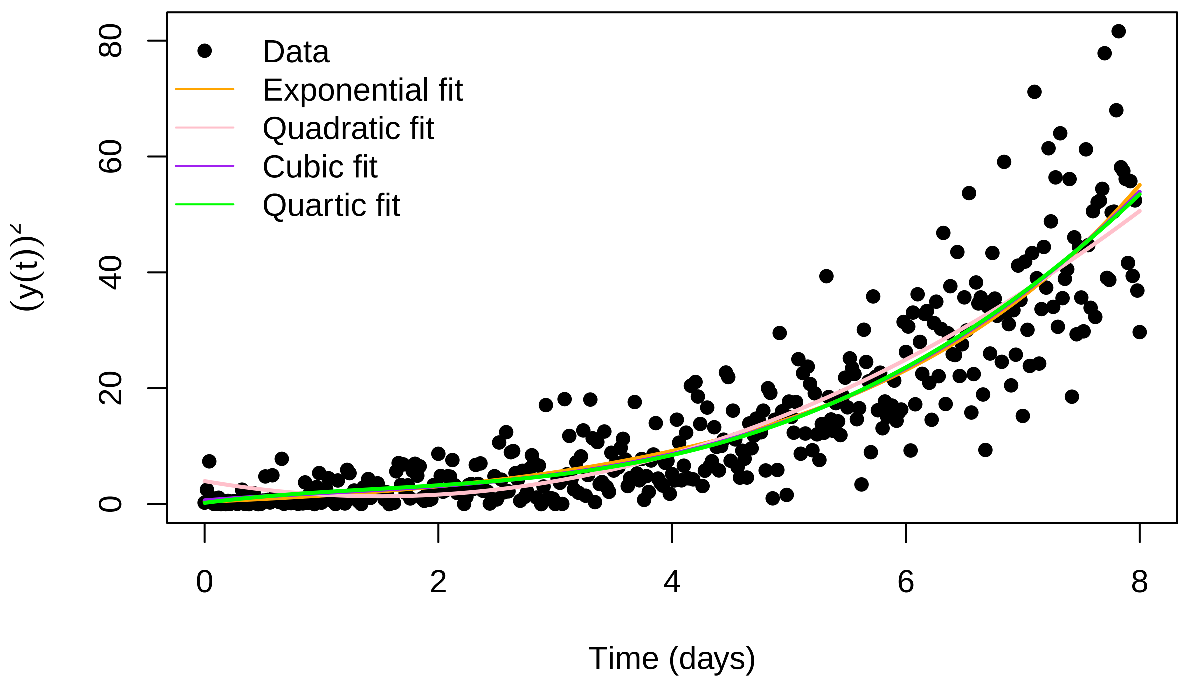
```
## Mechanistic Exponential   Quadratic       Cubic     Quartic
##    20501.58    20461.44    20998.40    20349.06    20340.19
```

```r
# Plotting all fitted curves
plot(data$t, data$z, pch = 16, xlab = "Time (days)",
     ylab = expression((y(t))^2))

lines(data$t[order(data$t)], data$exp_fitted[order(data$t)], lwd = 2,
      col = "orange")
lines(data$t[order(data$t)], data$poly2_fitted[order(data$t)], lwd = 2,
      col = "pink")
lines(data$t[order(data$t)], data$poly3_fitted[order(data$t)], lwd = 2,
      col = "purple")
lines(data$t[order(data$t)], data$poly4_fitted[order(data$t)], lwd = 2,
      col = "green")


legend("topleft", legend = c("Data", "Exponential fit", "Quadratic fit",
                             "Cubic fit", "Quartic fit"),
```

```r
       pch = c(16, NA, NA, NA, NA), lty = c(NA, 1, 1, 1, 1),
       col = c("black", "orange", "pink", "purple", "green"), bty = "n")
```



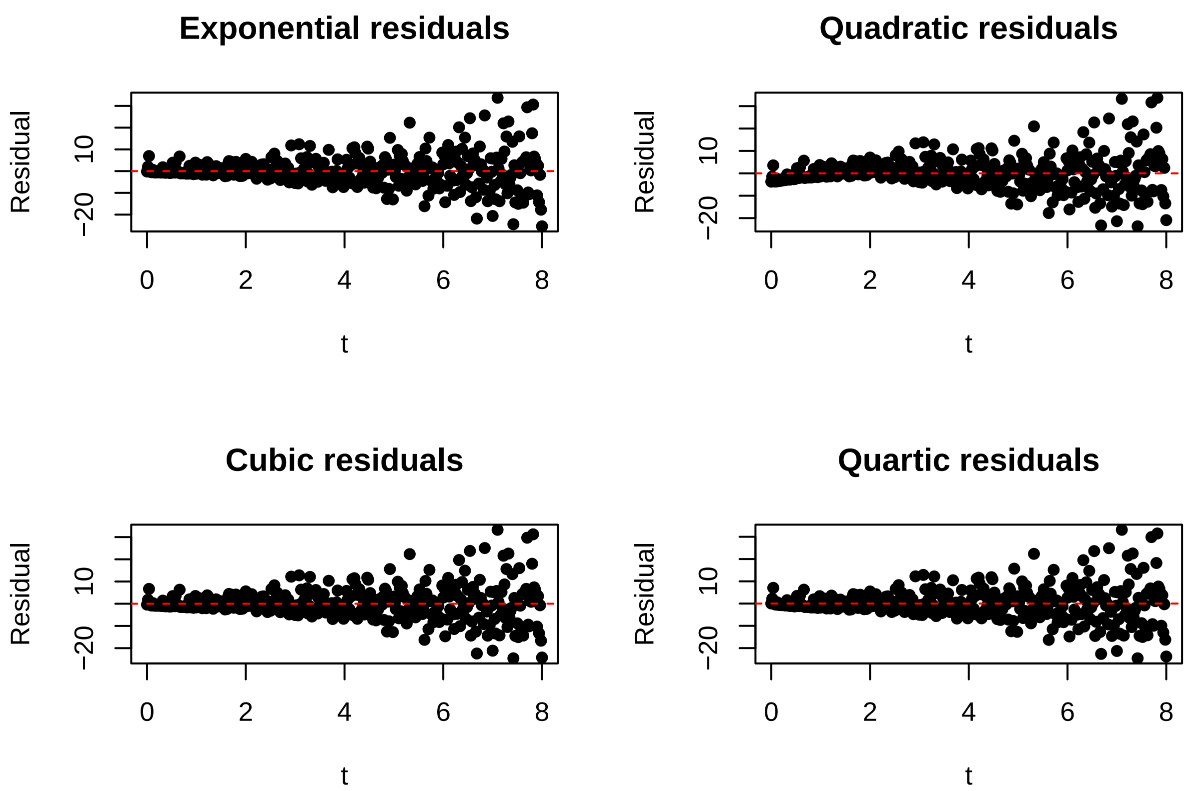

```r
# Plotting residuals side-by-side
par(mfrow = c(2, 2))

plot(data$t, data$z - data$exp_fitted, pch = 16,
     main = "Exponential residuals", xlab = "t", ylab = "Residual")
abline(h = 0, lty = 2, col = "red")

plot(data$t, data$z - data$poly2_fitted, pch = 16,
     main = "Quadratic residuals", xlab = "t", ylab = "Residual")
abline(h = 0, lty = 2, col = "red")

plot(data$t, data$z - data$poly3_fitted, pch = 16,
     main = "Cubic residuals", xlab = "t", ylab = "Residual")
abline(h = 0, lty = 2, col = "red")

plot(data$t, data$z - data$poly4_fitted, pch = 16,
     main = "Quartic residuals", xlab = "t", ylab = "Residual")
abline(h = 0, lty = 2, col = "red")
```




```r
par(mfrow = c(1, 1))
```